



32-Bit MIPS Pipeline With Hazard Detection



Team Members :

- 1- Omar Hatem Hassan
- 2- Ahmed Ebrahim Mohamed
- 3- Mohamed Samir Fathy
- 4- Merna Hesham Emam
- 5- Kholoud Ebrahim Hassanen

Under Supervision Of DR/ **Hatem Zakaria**

Table of Contents

1. Introduction
2. Architecture
3. From Single-Cycle to Pipeline
4. Hazards and Stalling
5. Pipeline Components
 - A. Pipeline Registers (The Stage Isolation Barriers)
 - B. The Forwarding Unit (Data Hazard Resolution)
 - C. The Hazard Detection Unit (Load-Use Interlocking)
 - D. Pipeline Flushes (Control Hazard & Exception Management)
6. Instruction Set
7. Project Output
8. Example of a Stall Condition from the Simulation Run
9. Example of a Flush Condition from the Simulation Run
10. References

Introduction

- **Core Objective:** The project involves designing a **32-bit MIPS Processor** that transitions from a slow single-cycle execution to a **5-stage pipeline** (IF → ID → EX → MEM → WB). This architectural technique overlaps multiple instructions simultaneously to maximize instruction throughput.

$$\text{Ideal Speedup Formula: } \textit{Speedup} = \frac{\textit{Execution Time}_{\textit{unpipelined}}}{\textit{Execution Time}_{\textit{pipelined}}}$$

- **The Challenge (Pipeline Hazards):** Pipelining inherently introduces structural, data, and control conflicts (hazards). If left unresolved, these hazards cause the processor to read stale data or execute incorrect instruction paths, compromising hardware integrity.
- **The Solution (Hazard Hardware):** To guarantee architectural correctness, a hardware-level **Hazard Detection and Forwarding Unit** is integrated to handle:
 - **Data Hazards (Load-Use Delay):** Mitigated via **Data Forwarding (Bypassing)** to route active ALU results instantly, or by inserting a **Stall (Bubble)** to freeze the pipeline when necessary.
 - **Control Hazards (Branches/Jumps):** Resolved by **Pipeline Flushing** to safely clear speculatively fetched instructions if a branch is taken or an exception occurs.

Pipeline Effect:

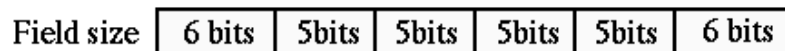
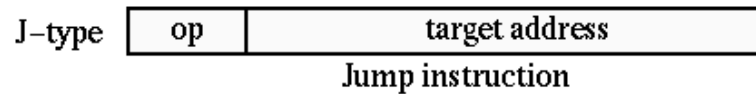
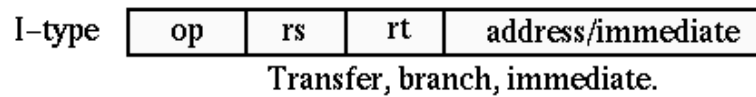
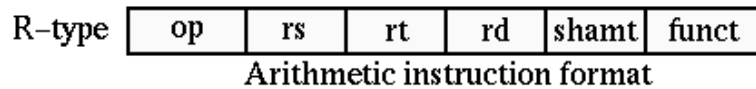
- **Throughput over Latency:** Pipelining does not make a single instruction execute faster (latency actually increases slightly due to pipeline registers). Instead, it drastically increases the total number of instructions completed per second (throughput).
- **Faster Clock Speeds:** The clock cycle is determined by the single slowest pipeline stage, rather than the time it takes to execute an entire instruction. This allows for significantly higher clock frequencies.
- **CPI (Cycles Per Instruction) Decreases:** While a single-cycle processor takes 1 long cycle per instruction, a pipeline aims for an ideal CPI of 1 at a much faster clock rate. However, real-world stalls (bubbles) from hazards will increase the actual CPI above 1.
- **Increased Hardware Costs:** Pipelining requires extra physical area on the chip for pipeline registers (to hold state between stages) and complex hazard/forwarding logic.
- **Higher Power Consumption:** The addition of more hardware registers and a faster clock frequency inherently draws more power.

Architecture

Instructions set Architecture (ISA)

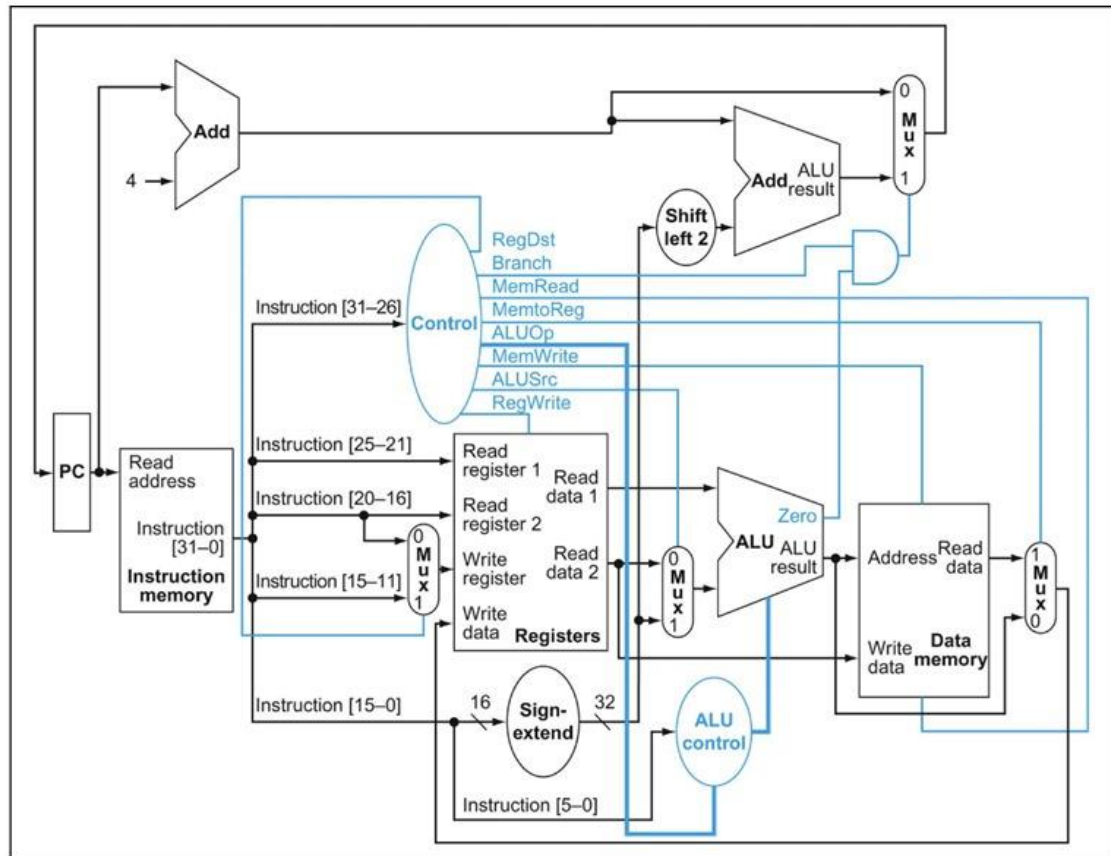
The datapath adapts dynamically based on three primary instruction formats decoded in the ID stage:

1. R-Type Instructions (e.g., add, sub, and)
2. I-Type Instructions (e.g., lw, sw, beq, addi)
3. J-Type Instructions (Jump) (e.g., j, jal)



32-Bit MIPS Pipeline With Hazard Detection

4.8 Single Cycle Implementation



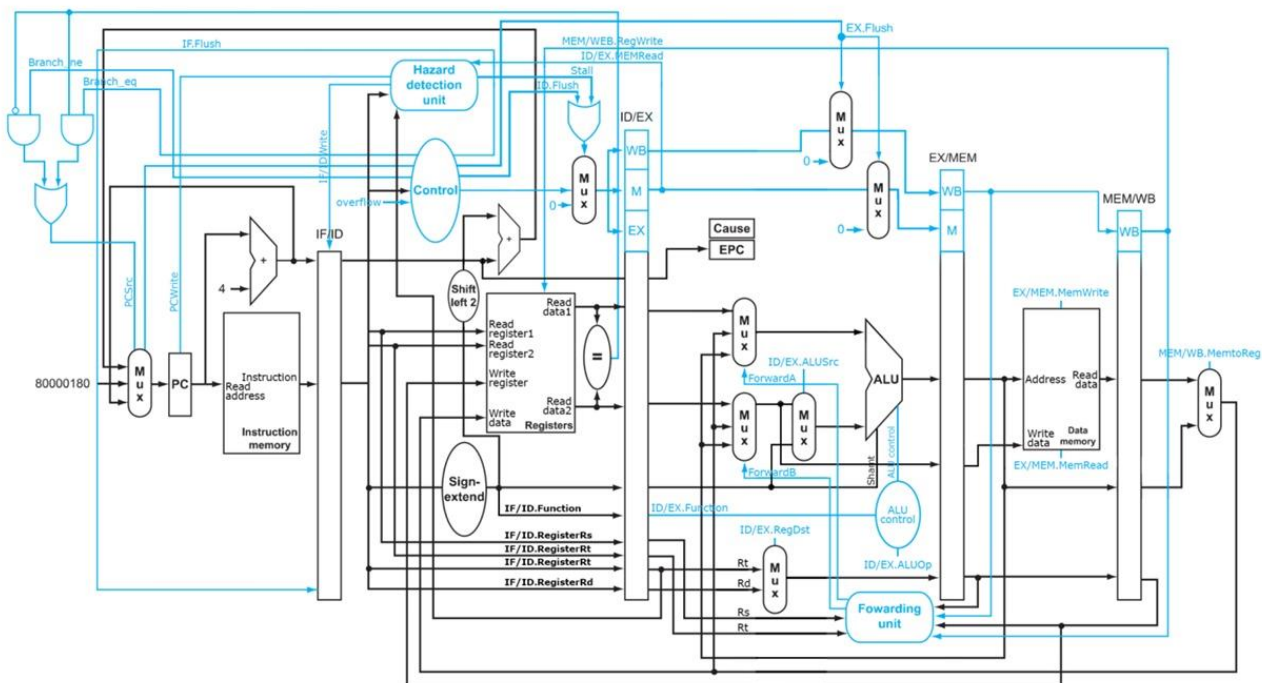
Pipelining does not make a single instruction execute faster (latency remains roughly the same), but it **increases throughput (the number of instructions completed per second)**.

- It treats the processor like a factory assembly line. The 32-bit architecture is split into 5 independent stages: **IF** (Fetch), **ID** (Decode), **EX** (Execute), **MEM** (Memory), and **WB** (Write-Back).
- As soon as an instruction moves from Stage 1 to Stage 2, the next instruction immediately enters Stage 1. At peak performance, **one instruction finishes every single clock cycle**, and the clock speed can be much faster because it only needs to wait for the single slowest *stage*, not the whole instruction path.

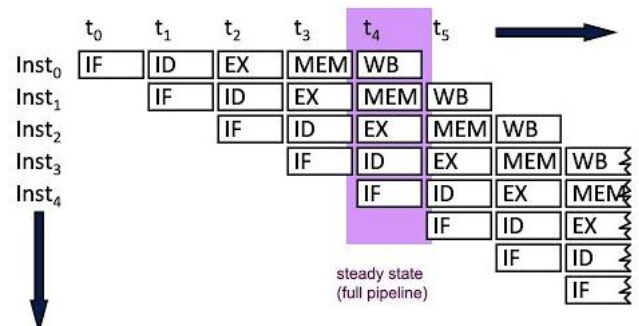
What We Add to a Single-Cycle Datapath to Create a Pipeline

To transform a single-cycle design into the pipelined architecture shown in your blueprint, we must inject specific hardware control networks and state-holding registers.

- A. Pipeline Registers (The Stage Isolation Barriers)
- B. The Forwarding Unit (Data Hazard Resolution)
- C. The Hazard Detection Unit (Load-Use Interlocking)
- D. Pipeline Flushes (Control Hazard & Exception Management)



Illustrating Pipeline Operation: Operation View



A. Pipeline Registers (The Stage Isolation Barriers)

Pipeline registers are placed between every pipeline stage to hold instruction data and control signals during each clock cycle.

Main pipeline registers in MIPS are:

- **IF/ID** → stores fetched instruction and PC+4
- **ID/EX** → stores decoded data, register values, immediate values, and control signals
- **EX/MEM** → stores ALU result, branch information, and memory control signals
- **MEM/WB** → stores data read from memory or ALU result before write back

Without these registers, stages would overwrite each other's data, making pipelining impossible.

B. The Forwarding Unit (Data Hazard Resolution)

The forwarding unit minimizes RAW (Read After Write) hazards by bypassing data directly from later stages to the EX stage instead of waiting for write-back.

Example:

```
add $t0, $t1, $t2
sub $t3, $t0, $t4
```

sub needs \$t0 before it is written back.

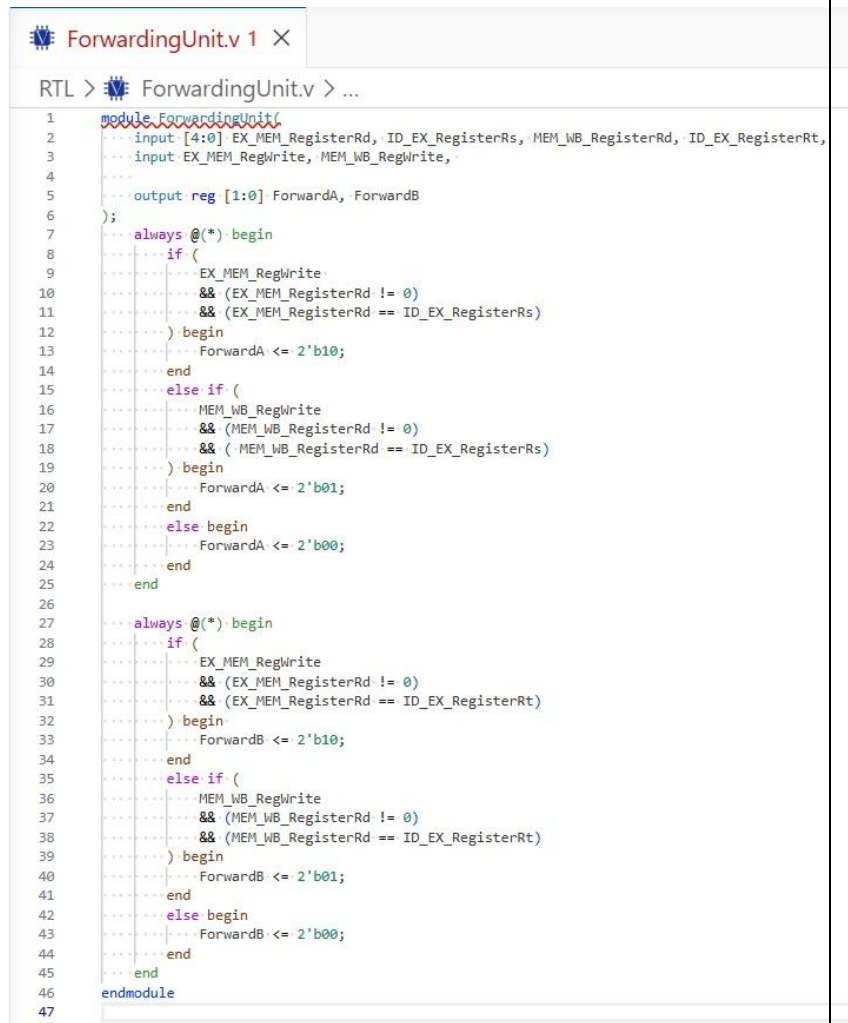
The forwarding unit forwards the ALU result from

EX/MEM or MEM/WB directly to the ALU input.

The unit compares:

- Source registers (R_s , R_t)
- Destination registers (R_d)
- RegWrite signals

and selects forwarded data using multiplexers.



```
RTL > ForwardingUnit.v > ...
1 module ForwardingUnit(
2     input [4:0] EX_MEM_RegisterRd, ID_EX_RegisterRs, MEM_WB_RegisterRd, ID_EX_RegisterRt,
3     input EX_MEM_RegWrite, MEM_WB_RegWrite,
4
5     output reg [1:0] ForwardA, ForwardB
6 );
7     always @(*) begin
8         if (
9             EX_MEM_RegWrite
10            && (EX_MEM_RegisterRd != 0)
11            && (EX_MEM_RegisterRd == ID_EX_RegisterRs)
12        ) begin
13            ForwardA <= 2'b10;
14        end
15        else if (
16            MEM_WB_RegWrite
17            && (MEM_WB_RegisterRd != 0)
18            && (MEM_WB_RegisterRd == ID_EX_RegisterRs)
19        ) begin
20            ForwardA <= 2'b01;
21        end
22        else begin
23            ForwardA <= 2'b00;
24        end
25    end
26
27    always @(*) begin
28        if (
29            EX_MEM_RegWrite
30            && (EX_MEM_RegisterRd != 0)
31            && (EX_MEM_RegisterRd == ID_EX_RegisterRt)
32        ) begin
33            ForwardB <= 2'b10;
34        end
35        else if (
36            MEM_WB_RegWrite
37            && (MEM_WB_RegisterRd != 0)
38            && (MEM_WB_RegisterRd == ID_EX_RegisterRt)
39        ) begin
40            ForwardB <= 2'b01;
41        end
42        else begin
43            ForwardB <= 2'b00;
44        end
45    end
46 endmodule
47
```


C. The Hazard Detection Unit (Load-Use Interlocking)

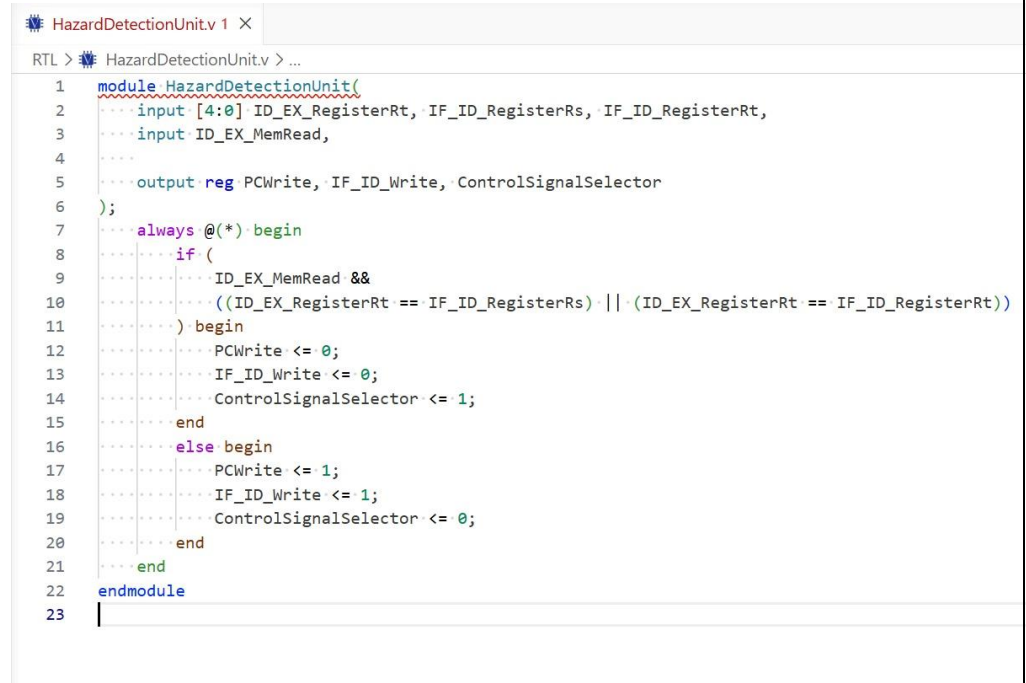
Forwarding cannot solve all hazards, especially the load-use hazard.

Example:

```
lw $t0, 0($t1)
add $t2, $t0, $t3
```

The data from `lw` becomes available only after MEM stage, so `add` cannot use it immediately. The hazard detection unit:

- Detects this dependency
- Freezes the PC and IF/ID register
- Inserts a NOP into ID/EX



```
1 module HazardDetectionUnit(
2     input [4:0] ID_EX_RegisterRt, IF_ID_RegisterRs, IF_ID_RegisterRt,
3     input ID_EX_MemRead,
4
5     output reg PCWrite, IF_ID_Write, ControlSignalSelector
6 );
7 always @(*) begin
8     if (
9         ID_EX_MemRead &&
10        ((ID_EX_RegisterRt == IF_ID_RegisterRs) || (ID_EX_RegisterRt == IF_ID_RegisterRt))
11    ) begin
12        PCWrite <= 0;
13        IF_ID_Write <= 0;
14        ControlSignalSelector <= 1;
15    end
16    else begin
17        PCWrite <= 1;
18        IF_ID_Write <= 1;
19        ControlSignalSelector <= 0;
20    end
21 end
22 endmodule
23
```

This creates a 1-cycle stall until the data is ready.

Condition commonly checked:

```
if (ID/EX.MemRead &&
    ((ID/EX.Rt == IF/ID.Rs) ||
     (ID/EX.Rt == IF/ID.Rt)))
```

D. Pipeline Flushes (Control Hazard & Exception Management)

Flushing clears invalid instructions from the pipeline by replacing them with NOPs.

Used in: (Branches \ Jumps \ Exceptions \ Interrupts)

Example:

```
beq $t0, $t1, TARGET
```

The processor may fetch the next sequential instruction before knowing whether the branch is taken.

If the branch is taken, those fetched instructions are wrong and must be flushed.

Flush operation:

- Clears control signals
- Converts instructions into NOPs
- Prevents incorrect state changes

This maintains correct execution flow in the pipeline.

Hazards

Pipeline Hazards are timing or resource conflicts that cause the processor to execute instructions using stale or incorrect data. This microarchitecture completely resolves them using two primary approaches:

1. Data Hazards (Read-After-Write)

A Data Hazard occurs when an instruction depends on the result of a previous instruction that has not yet completed its Write-Back (**WB**) stage.

- **Arithmetic Forwarding:** For data dependencies between arithmetic instructions (like `add`), data is ready in the **EX** stage but not yet written to the Register File. The **Forwarding Unit** intercepts this data from the EX/MEM or MEM/WB pipeline registers and routes it directly to the ALU inputs via ForwardA and ForwardB multiplexer lines, eliminating performance loss.
- **Load-Use Dependency:** If a `lw` instruction is immediately followed by an instruction that depends on its loaded data, forwarding alone is physically impossible because memory data is not available until the end of the **MEM** stage. This specific case strictly requires a **1-cycle hardware stall**.

2. Control Hazards (Instruction Flow Disruptions)

Control Hazards occur during conditional branches (`beq`, `bne`) and jumps (`j`), where the processor speculatively fetches subsequent instructions ($PC + 4$) before knowing the actual branch outcome.

- **Optimized Early Resolution:** To minimize performance penalties, this design optimizes branches by resolving them early in the **ID Stage** using a dedicated hardware comparator (`==`). If a branch evaluates to true, the hardware activates **IF.Flush** to clear the incorrectly fetched instruction inside the IF/ID register, converting it into a harmless NOP (No-Operation) to protect processor states.

Stalling

When a **Load-Use Data Hazard** is detected, the **Hazard Detection Unit** in the ID stage evaluates the following Boolean logic expression:

```
if (
    ID_EX_MemRead &&
    ((ID_EX_RegisterRt == IF_ID_RegisterRs) || (ID_EX_RegisterRt == IF_ID_RegisterRt))
)
```

If true, the unit enforces a synchronized 1-cycle freeze across the early pipeline stages:

1. **PCWrite = 0:** Freezes the Program Counter register, keeping it pointed at the same instruction for the next cycle.
2. **IF/IDWrite = 0:** Freezes the IF/ID pipeline register, keeping the dependent instruction held in the Decode stage.
3. **Control Signal Muxing:** Forces all control signals (such as `RegWrite`, `MemWrite`) entering the ID/EX register to 0.

The instruction set

This code stores six numbers in an array in the memory then it loops to check the MAX, MIN and the sum of all elements [5, 9, 19, 12, 1, 15].

```
26  */
27  N_O_P          // 00000000 (1)
28
29  addi t2 $zero 0x5 // 200A0005 (2) max
30  addi t3 $zero 0x5 // 200B0005 (3) min
31  addi s0 $zero 0x5 // 20100005 (4) sum
32
33  //// Storing elements in the array ////
34  addi t7 $zero 0x5 // 200F0005 (5)
35  sw t7 0x0 (t0)    // AD0F0000 (6)
36
37  addi t7 $zero 0x9 // 200F0009 (7)
38  sw t7 0x4 (t0)    // AD0F0004 (8)
39
40  addi t7 $zero 0x19 // 200F0019 (9)
41  sw t7 0x8 (t0)    // AD0F0008 (10)
42
43  addi t7 $zero 0x12 // 200F0012 (11)
44  sw t7 0xC (t0)    // AD0F000C (12)
45
46  addi t7 $zero 0x1 // 200F0001 (13)
47  sw t7 0x10 (t0)   // AD0F0010 (14)
48
49  addi t7 $zero 0x15 // 200F0015 (15)
50  sw t7 0x14 (t0)   // AD0F0014 (16)
51  //////////////////////////////////////
52
53  // for loop
54      addi t1 $zero 0x1 // 20090001 (17) i = 1
55      addi t6 $zero 0x6 // 200E0006 (18) arr size = 6
56
57      // Loading element from memory
58  loop: addi t0 t0 0x4 // 21080004 (19)
59      lw t5 0x0 (t0) // 8D0D0000 (20) // t5 = arr[i]
60      add s0 s0 t5 // 020D8020 (21) // sum += arr[i]
61      slt t8 t2 t5 // 014DC02A (22) // if(t2 < t5) => t8 = 1;
62      slt t9 t5 t3 // 01ABC82A (23) // if(t5 < t3) => t9 = 1;
63      addi t1 t1 0x1 // 21290001 (24) // i = i + 1;
64      beq t8 $zero NotMax // 13000001 (25)
65      add t2 $zero t5 // 000D5020 (26) // max = arr[i]
66  NotMax: beq t9 $zero NotMin // 13200001 (27)
67      add t3 $zero t5 // 000D5820 (28) // min = arr[i]
68  NotMin: bne t1 t6 loop // 152EFFF5 (29) // loop condition (i < arr.size)
```

Memory Data - /testbench/processor/DPath/dataMem - **DataMem**

00000074	00000000	00000000	00000000	00000000
00000078	00000000	00000000	00000000	00000000
0000007c	00000005	00000000	00000000	00000000
00000080	00000009	00000000	00000000	00000000
00000084	00000019	00000000	00000000	00000000
00000088	00000012	00000000	00000000	00000000
0000008c	00000001	00000000	00000000	00000000
00000090	00000015	00000000	00000000	00000000
00000094	00000000	00000000	00000000	00000000
00000098	00000000	00000000	00000000	00000000
0000009c	00000000	00000000	00000000	00000000
000000a0	00000000	00000000	00000000	00000000
000000a4	00000000	00000000	00000000	00000000
000000a8	00000000	00000000	00000000	00000000

The array of elements

Memory Data - /testbench/processor/DPath/regFile/regmem -

0000001f	00000000
0000001e	00000000
0000001d	00000000
0000001c	00000000
0000001b	00000000
0000001a	00000000
00000019	00000000
00000018	00000000
00000017	00000000
00000016	00000000
00000015	00000000
00000014	00000000
00000013	00000000
00000012	00000000
00000011	00000000
00000010	0000004f
0000000f	00000015
0000000e	00000006
0000000d	00000015
0000000c	00000000
0000000b	00000001
0000000a	00000019
00000009	00000006
00000008	00000094
00000007	00000000

Sum at \$s0

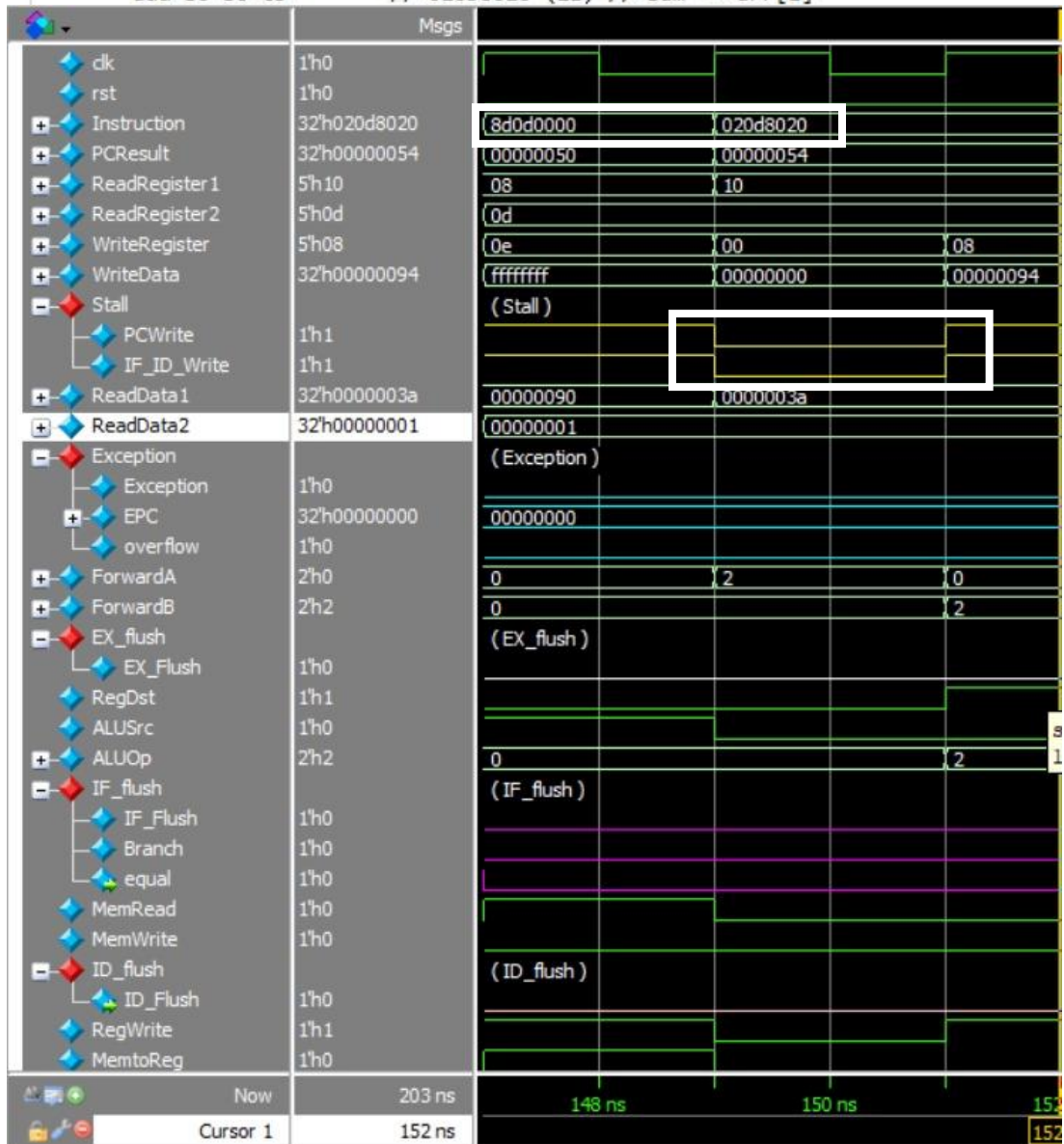
Min at \$t3

Max at \$t2

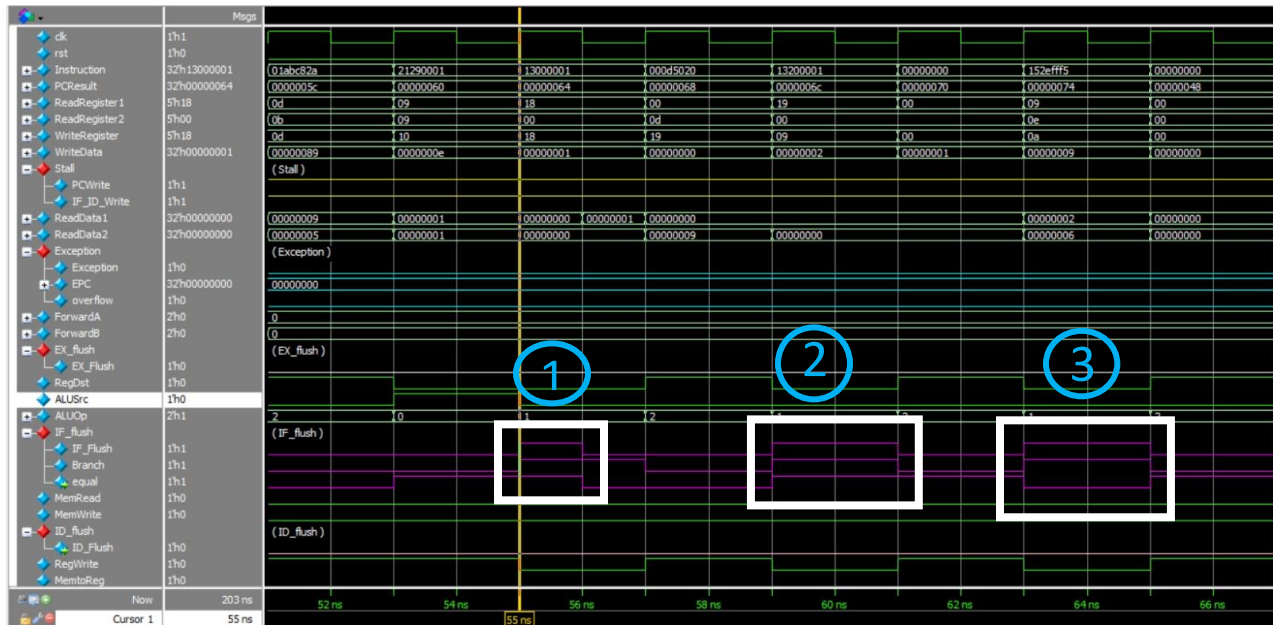
The results

Example of a stall condition from the run

```
// Loading element from memory
loop:  addi t0 t0 0x4      // 21080004 (19)
      lw t5 0x0 (t0)     // 8D0D0000 (20) // t5 = arr[i]
      add s0 s0 t5       // 020D8020 (21) // sum += arr[i]
```



Example of a flush condition from the run



Iteration 1 (Value = 9):

- **Check Max:** `9 > 5` . It is a new max!
 - `beq t8 $zero NotMax` (25) is **Not Taken**. (No Flush). 1
- **Check Min:** `9 < 5` is False.
 - `beq t9 $zero NotMin` (27) is **Taken**. 2
 - → **IF_Flush Triggers!** (Flushes instruction 28).
- **Loop End:** `t1 (2) != 6`.
 - `bne t1 t6 loop` (29) is **Taken**. 3
 - → **IF_Flush Triggers!**

Reference :

Computer Organization and Design- The HW_SW Interface 5th edition -
David A. Patterson, John L. Hennessy